

UNIVERSIDAD
[PENDING: NOMBRE DE LA UNIVERSIDAD]

Aseguramiento de una API de Música mediante OAuth 2.0 y JSON Web Tokens: el caso SoundAccess

Informe académico — Semana 7
Autenticación y Autorización con OAuth 2.0 / JWT

Presentado por
[PENDING: FULL NAME]

Carné / ID de estudiante	[PENDING: STUDENT ID]
Sede / Sección	[PENDING: CAMPUS / SECTION]
Tecnología principal	Python · FastAPI · SQLAlchemy · SQLite · PyJWT · Argon2
Repositorio	[PENDING: REPOSITORY URL]

Guatemala, agosto de 2026

Tabla de contenido

1. Introducción.....	3
2. Objetivos.....	3
2.1 Objetivo general.....	3
2.2 Objetivos específicos.....	3
3. Marco conceptual.....	4
4. Desarrollo de la actividad.....	5
5. Arquitectura del sistema.....	5
6. Diseño de la base de datos.....	6
7. Implementación de OAuth 2.0.....	7
8. Authorization Code + PKCE.....	7
9. Client Credentials.....	8
10. Implementación de JWT.....	9
11. Scopes y autorización.....	9
12. Recursos protegidos.....	9
13. Pruebas y resultados.....	10
14. Controles de seguridad.....	11
15. Problemas encontrados y soluciones.....	11
16. Conclusiones.....	12
17. Recomendaciones.....	12
18. Referencias.....	13
19. Apéndices.....	13
19.1 Matriz de requisitos, evidencia y estado.....	13
19.2 Diagramas fuente.....	14

1. Introducción

Los servicios de streaming e integraciones musicales modernas rara vez exponen la contraseña de un usuario a las aplicaciones de terceros; en su lugar delegan el acceso mediante un protocolo de autorización. Este informe documenta el diseño, la implementación y la verificación de SoundAccess, una API de catálogo musical construida como práctica académica para aplicar OAuth 2.0 (RFC 6749) y JSON Web Tokens (RFC 7519) en un escenario realista: un cliente de usuario que necesita leer y escribir playlists en nombre de una persona, y un cliente de servicio que sincroniza el catálogo sin intervención humana.

El proyecto no es una simulación superficial de OAuth: implementa un servidor de autorización que emite y valida tokens firmados, un servidor de recursos que aplica scopes por endpoint y verifica la propiedad de los recursos, y un cliente de demostración que ejecuta el flujo completo — incluido PKCE — desde el navegador. Todo el desarrollo se realizó con Claude Code como herramienta principal de ingeniería (creación de archivos, pruebas, depuración y control de versiones), bajo la orquestación de Claude Cowork.

2. Objetivos

2.1 Objetivo general

Diseñar, implementar y verificar una API de música protegida mediante OAuth 2.0 y JWT, demostrando de forma práctica la diferencia entre autenticación y autorización en un sistema con múltiples tipos de cliente.

2.2 Objetivos específicos

1. Implementar el flujo Authorization Code con PKCE (S256) para un cliente público de usuario final.
2. Implementar el flujo Client Credentials para un cliente de servicio (machine-to-machine).
3. Emitir y validar JSON Web Tokens con los claims exigidos por RFC 9068, rechazando algoritmos inseguros.
4. Aplicar autorización basada en scopes a nivel de Servidor de Recursos, no solo declararla en el token.
5. Garantizar el aislamiento de recursos entre usuarios (una playlist privada no es accesible por otro usuario, con un token válido).
6. Cubrir los seis escenarios obligatorios y casos adicionales de endurecimiento con pruebas automatizadas.

7. Producir documentación técnica, diagramas editables y evidencia real de ejecución.

3. Marco conceptual

OAuth 2.0 (RFC 6749) define cuatro roles: el propietario del recurso (resource owner, típicamente una persona), el cliente (client, la aplicación que solicita acceso), el servidor de autorización (authorization server, que autentica y emite tokens) y el servidor de recursos (resource server, que expone la API protegida). El protocolo no transporta contraseñas hacia el cliente: en su lugar, el cliente obtiene un token de acceso de vida corta con un alcance (scope) delimitado.

El flujo Authorization Code es el recomendado para clientes que interactúan con un usuario a través de un navegador. Cuando el cliente es público — es decir, no puede guardar un secreto de forma confidencial, como una aplicación web servida al navegador — RFC 7636 introduce PKCE (Proof Key for Code Exchange): el cliente genera un valor aleatorio (code_verifier), deriva un code_challenge = BASE64URL(SHA-256(code_verifier)) y lo envía en la solicitud de autorización; al canjear el código, debe volver a presentar el code_verifier original, que el servidor recalcula y compara. Esto evita que un atacante que intercepte el código de autorización pueda canjearlo sin conocer también el verifier.

El flujo Client Credentials (RFC 6749 §4.4) es apropiado para comunicación servicio-a-servicio, sin un usuario presente: el cliente se autentica directamente con su client_id y client_secret y recibe un token que representa al propio cliente, no a una persona.

Un JSON Web Token (RFC 7519) es una estructura firmada (en este proyecto, HMAC-SHA256) que transporta claims — pares clave/valor — de forma verificable. RFC 9068 estandariza el perfil de JWT para tokens de acceso de OAuth 2.0, especificando claims como iss (emisor), sub (sujeto), aud (audiencia), exp (expiración), iat (emisión) y client_id. RFC 9700 recopila las mejores prácticas de seguridad vigentes para OAuth 2.0, incluyendo la recomendación de exigir PKCE también en clientes confidenciales y evitar el grant de Resource Owner Password Credentials (ROPC), que expone la contraseña del usuario directamente al cliente.

Un punto conceptual central del proyecto es la distinción entre autenticación y autorización: autenticar responde «quién eres» (¿la firma del token es válida?, ¿el emisor y la audiencia son correctos?), mientras que autorizar responde «qué puedes hacer» (¿el token incluye el scope necesario?, ¿el recurso solicitado te pertenece?). Un token perfectamente válido puede, aun así, ser insuficiente para una operación concreta.

4. Desarrollo de la actividad

El trabajo se organizó en fases incrementales, cada una cerrada con una batería de verificación antes de avanzar a la siguiente, y con commits de Git independientes por fase (ver Sección 15 y el historial real del repositorio).

8. Reconocimiento del entorno: se confirmó la disponibilidad de Python 3.11, Node.js, Git, un navegador Chromium preinstalado y las herramientas de prueba; se protegió el directorio Documents del usuario creando un directorio de proyecto dedicado (Week7_SoundAccess_OAuth_API) sin tocar archivos preexistentes.
9. Fundamento del proyecto: inicialización de Git, entorno Python, dependencias (FastAPI, SQLAlchemy, PyJWT, argon2-cffi, pytest) y gestión de configuración mediante variables de entorno (.env, nunca comprometido a Git).
10. Base de datos: modelado de las seis entidades requeridas, hashing de contraseñas y secretos con Argon2id, y un script de siembra idempotente con dos usuarios, dos clientes OAuth y datos ficticios de catálogo.
11. Identidad y OAuth: registro controlado de clientes, endpoint de autorización con validación estricta de redirect_uri, y los flujos Authorization Code + PKCE y Client Credentials.
12. JWT y API de recursos: emisión y validación de tokens, dependencias de FastAPI para exigir scopes, y los cinco endpoints protegidos mínimos.
13. Cliente de demostración: interfaz mínima en HTML/JS que ejecuta PKCE en el navegador (generación de code_verifier, cálculo SHA-256 vía Web Crypto API) y consume la API con el token obtenido.
14. Pruebas automatizadas: 38 pruebas con pytest, incluidos los seis escenarios obligatorios y pruebas de endurecimiento adicionales.
15. Documentación y evidencia: diagramas Mermaid editables, captura de evidencia real (peticiones HTTP reales y automatización de navegador con Playwright), README técnico y este informe.
16. Endurecimiento y verificación final: auditoría de seguridad dirigida, verificación contra la rúbrica y prueba de reproducibilidad desde cero.

5. Arquitectura del sistema

El sistema separa tres roles lógicos: el Cliente OAuth (SoundAccess Web Player, servido desde frontend/), el Servidor de Autorización (app/oauth) y el Servidor de Recursos (app/api), además de la base de datos compartida. Por simplicidad de despliegue local, los tres componentes se ejecutan dentro de un mismo proceso FastAPI, pero cada uno vive en un módulo independiente con sus propias

dependencias y responsabilidades — el Servidor de Recursos, por ejemplo, no conoce el mecanismo de login; solo valida el JWT que recibe.

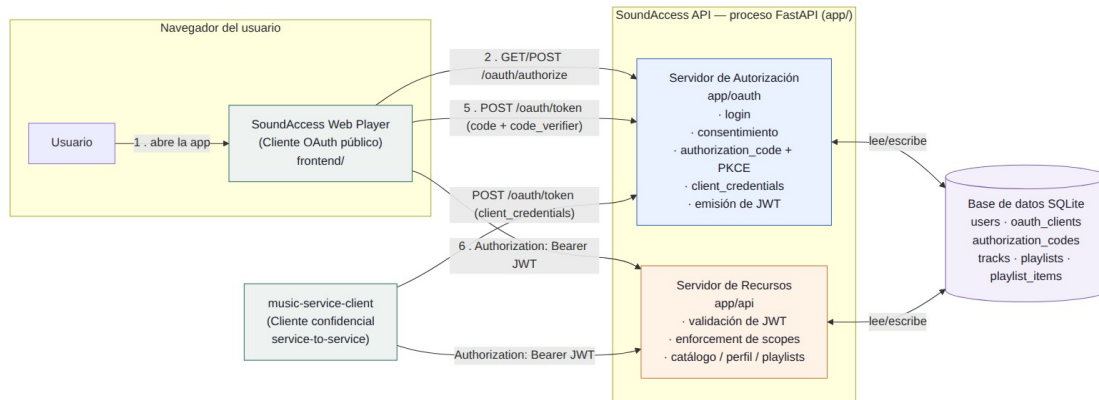


Figura 1. Diagrama de componentes (fuente editable en docs/diagrams/component_diagram.mmd).

6. Diseño de la base de datos

Se definieron seis entidades con SQLAlchemy 2.0, usando SQLite para el desarrollo local (persistencia real, no una base de datos en memoria) manteniendo la capa de acceso lo suficientemente abstracta para migrar a otro motor relacional si fuera necesario.

Entidad	Propósito	Notas de seguridad
users	Usuarios finales de la aplicación	password_hash con Argon2id; nunca texto plano
oauth_clients	Cientes OAuth registrados	client_secret_hash (Argon2id) o NULL en clientes públicos
authorization_codes	Códigos de autorización emitidos	Se guarda solo el hash SHA-256 del código; un solo uso; expira en 60s
tracks	Catálogo musical ficticio	Sin datos con derechos de autor de terceros
playlists	Playlists de un usuario	Ligada a owner_id; consultada siempre filtrando por dueño
playlist_items	Canciones dentro de una playlist	Restricción de unicidad (playlist_id, position)

Tabla 1. Entidades principales del modelo de datos.

7. Implementación de OAuth 2.0

El registro de clientes (POST /oauth/clients) es controlado: exige un header X-Registration-Key con una clave administrativa definida por variable de entorno; no existe una ruta pública que permita crear clientes privilegiados sin autenticación. Se sembraron dos clientes de demostración: web-user-client (público, sin secreto, obligado a usar PKCE) y music-service-client (confidencial, con secreto almacenado como hash Argon2id, restringido al scope catalog:read).

El endpoint de autorización (GET/POST /oauth/authorize) valida el client_id y compara el redirect_uri contra la lista blanca registrada mediante coincidencia exacta de cadena. Si el cliente o el redirect_uri no son válidos, el servidor no redirige — renderiza una página de error local — precisamente para no convertirse en un mecanismo de redirección abierta (open redirect) que un atacante pudiera aprovechar.

8. Authorization Code + PKCE

El flujo completo, tal como se implementó y se verificó con Playwright contra la aplicación en ejecución real:

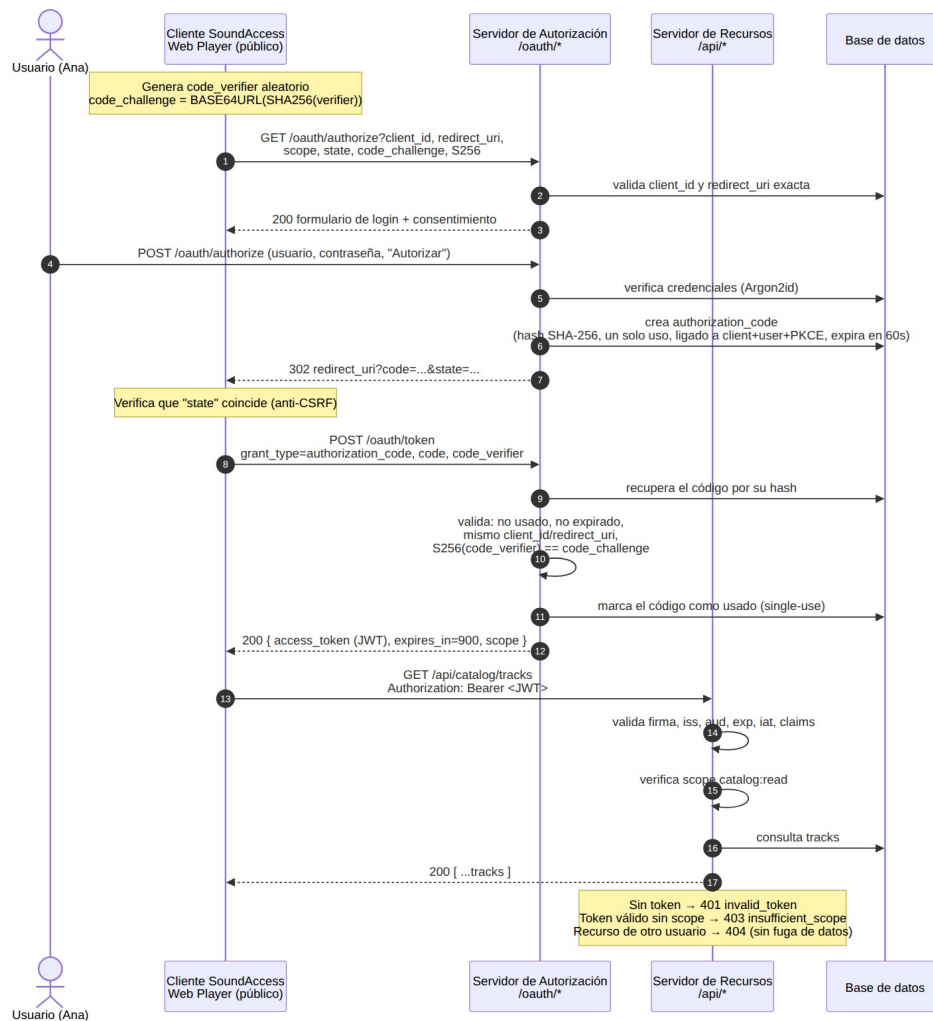


Figura 2. Secuencia completa Authorization Code + PKCE (fuente editable en docs/diagrams/sequence_diagram.mmd).

El código de autorización se almacena únicamente como su hash SHA-256 (nunca en texto plano), tiene una vida de 60 segundos, está ligado al cliente, al usuario, al redirect_uri exacto y al code_challenge, y se marca como usado antes de emitir el token — de modo que un intento de reutilización (replay) es rechazado con invalid_grant. El método code_challenge_method=plain es explícitamente rechazado; solo se acepta S256.

9. Client Credentials

El cliente de servicio se autentica con client_id y client_secret; el servidor verifica el secreto contra el hash Argon2id almacenado y valida que el scope solicitado esté dentro de lo permitido para ese cliente. El token resultante tiene sub == client_id (representa al cliente, no a una persona) y, aunque es un token válido, es

rechazado con 403 al solicitar `/api/me` o cualquier playlist privada — la aplicación demuestra explícitamente que autenticación válida no implica acceso ilimitado.

10. Implementación de JWT

Los tokens de acceso incluyen los claims `iss`, `sub`, `aud`, `exp`, `iat`, `jti`, `client_id` y `scope`. La validación (PyJWT) fija explícitamente el algoritmo permitido mediante el parámetro `algorithms=[...]`, lo que hace que un token con `alg=none` o con un algoritmo distinto al configurado sea rechazado antes de cualquier verificación de firma — la biblioteca nunca infiere el algoritmo a partir del propio token. Se validan además `issuer`, `audience`, `expiración`, `emisión` y la presencia de todos los claims requeridos. Los tokens tienen una vida de 15 minutos y solo se aceptan en el header `Authorization: Bearer`, nunca en la URL.

11. Scopes y autorización

Scope	Efecto
<code>catalog:read</code>	Leer el catálogo público de canciones
<code>profile:read</code>	Leer el perfil del usuario autenticado (solo tokens de usuario)
<code>playlist:read</code>	Leer las playlists propias
<code>playlist:write</code>	Crear y eliminar playlists propias

Tabla 2. Scopes definidos y su efecto.

Los scopes se comprueban en el Servidor de Recursos mediante una dependencia de FastAPI (`require_scopes`) aplicada a cada endpoint; no se limitan a figurar en el token.

12. Recursos protegidos

Método	Ruta	Scope	Comportamiento
GET	<code>/api/catalog/tracks</code>	<code>catalog:read</code>	Lista el catálogo ficticio
GET	<code>/api/me</code>	<code>profile:read</code>	Perfil del usuario (rechaza tokens de servicio)
POST	<code>/api/playlists</code>	<code>playlist:write</code>	Crea una playlist del usuario autenticado
GET	<code>/api/playlists/{id}</code>	<code>playlist:read</code>	Solo si el usuario es el dueño
DELETE	<code>/api/playlists/{id}</code>	<code>playlist:write</code>	Solo si el usuario es el

Método	Ruta	Scope	Comportamiento
			dueño

Tabla 3. Endpoints protegidos mínimos.

Semántica de errores: 401 cuando la solicitud no pudo autenticarse (token ausente, malformado, firma inválida, expirado, issuer o audience incorrectos); 403 cuando el token es válido pero el sujeto no está autorizado (scope faltante, o un token de servicio sobre un recurso personal); 404 — en vez de 403 — para una playlist ajena, de modo que la respuesta no revele si el recurso existe.

13. Pruebas y resultados

Se implementaron 38 pruebas automatizadas con pytest y el TestClient de FastAPI (tests/test_scenarios.py y tests/test_security.py), ejecutadas contra una base de datos SQLite aislada y credenciales sintéticas generadas para la sesión de pruebas. Las 38 pruebas pasan de forma consistente (ver docs/evidence/ev10_pytest_results.txt).

Escenario obligatorio	Resultado
1. Authorization Code + PKCE completo	200 en cada paso, token emitido y consumido
2. Client Credentials	Token emitido; acceso limitado al scope permitido
3. Solicitud protegida sin token	401 en los cinco endpoints protegidos
4. Token inválido / expirado / issuer o audience erróneos	401 en todos los casos
5. Token válido sin el scope requerido	403 insufficient_scope
6. Acceso de un usuario a la playlist de otro	404, sin fuga de datos privados

Tabla 4. Resultado de los seis escenarios obligatorios.

Las pruebas adicionales de endurecimiento cubren: PKCE inválido, reutilización de código, código expirado, redirect_uri no coincidente en el canje, secreto de cliente incorrecto, escalamiento de scope en client_credentials, grant ROPC no soportado, alg=none, claims requeridos ausentes, token en query string, registro de clientes sin clave administrativa, y una entrada tipo inyección SQL almacenada de forma inerte gracias al ORM.

14. Controles de seguridad

- Contraseñas de usuario y secretos de cliente con Argon2id; nunca en texto plano.
- Códigos de autorización almacenados solo como hash SHA-256, de un solo uso y expiración corta.
- Validación exacta de `redirect_uri`; ningún caso reencamina a un origen no registrado (sin open redirect).
- Algoritmo de firma JWT fijado explícitamente; `alg=none` y sustitución de algoritmo rechazados.
- CORS restringido a orígenes locales explícitos, sin comodín.
- Manejador global de excepciones: ninguna traza de pila se expone al cliente.
- Sin grant ROPC (contraseña de usuario nunca llega directamente al cliente).
- Documentación Swagger UI auto-hospedada (sin dependencia de un CDN externo).
- Revisión manual de logs de desarrollo para confirmar que no se registran contraseñas, secretos ni tokens.

15. Problemas encontrados y soluciones

Esta sección describe dificultades reales surgidas durante la implementación, no hipotéticas.

15.1 Dependencia faltante para validación de correo electrónico

Al definir el esquema `ProfileOut` con un campo `EmailStr`, Pydantic requiere el paquete `email-validator`, que no estaba instalado inicialmente y provocó un error de importación al iniciar la aplicación. Se resolvió instalando `pydantic[email]` y agregándolo a `requirements.txt`.

15.2 Fuga de detalle interno en el mensaje de error 401

La primera versión de la validación de tokens propagaba el texto crudo de la excepción de PyJWT (incluyendo, en un caso, un mensaje de error de decodificación de códec) directamente al cliente. Aunque no era una traza de pila ni exponía secretos, no era una práctica limpia. Se corrigió registrando la causa exacta únicamente en el log del servidor y devolviendo al cliente un mensaje genérico y conforme a RFC 6750.

15.3 Swagger UI en blanco por dependencia de un CDN externo

La configuración por defecto de FastAPI para `/docs` carga `swagger-ui-bundle.js` y `swagger-ui.css` desde `cdn.jsdelivr.net`. En el entorno de ejecución utilizado, ese

dominio no era accesible, por lo que la página de documentación se renderizaba en blanco. Se resolvió vendorizando los archivos de swagger-ui-dist dentro del propio repositorio (app/static/swagger-ui/) y sirviéndolos con FastAPI StaticFiles, de modo que la documentación interactiva funciona sin acceso a Internet — una mejora relevante para la reproducibilidad exigida en la Sección 33 del plan de trabajo.

15.4 Inestabilidad al reiniciar el servidor de desarrollo en la terminal automatizada

Al encadenar pkill, la recreación de la base de datos y el arranque de uvicorn en un mismo bloque de comandos, el proceso en segundo plano se interrumpía de forma intermitente. Se resolvió separando cada paso en comandos independientes y arrancando el servidor con setsid/disown para desacoplarlo del proceso padre de la terminal.

16. Conclusiones

17. Implementar los dos flujos de OAuth 2.0 de punta a punta — en vez de solo describirlos — hizo tangible por qué PKCE es indispensable para clientes públicos: sin él, cualquiera que interceptara el código de autorización podría canjearlo.
18. La distinción entre autenticación y autorización dejó de ser una definición de libro de texto en cuanto el token de client_credentials, siendo perfectamente válido, tuvo que ser rechazado explícitamente al pedir un recurso personal.
19. Diseñar primero las respuestas de error (401 frente a 403 frente a 404) obligó a pensar en qué información se filtra incluso en una denegación de acceso — la decisión de responder 404 en vez de 403 para una playlist ajena fue una consecuencia directa de ese análisis.
20. Vendorizar los recursos de Swagger UI resultó una lección práctica sobre reproducibilidad: una demostración que depende de un CDN externo puede fallar en un entorno de evaluación con red restringida, aunque el código en sí sea correcto.

17. Recomendaciones

- Incorporar refresh tokens con rotación y revocación para escenarios donde una sesión debe sobrevivir más allá de la vida corta del access token.
- Migrar la base de datos a PostgreSQL con Alembic para gestionar migraciones si el proyecto creciera más allá de una demostración local.
- Añadir límite de tasa (rate limiting) al endpoint de token y al de autorización para mitigar ataques de fuerza bruta sobre credenciales.

- Integrar un análisis de seguridad estático (por ejemplo, bandit) y un escaneo de dependencias en un flujo de integración continua.
- Evaluar el uso de claves asimétricas (RS256/ES256) en lugar de HMAC si en el futuro varios servicios necesitaran verificar tokens sin compartir un secreto simétrico.

18. Referencias

- IETF. RFC 6749 — The OAuth 2.0 Authorization Framework. <https://www.rfc-editor.org/rfc/rfc6749>
- IETF. RFC 6750 — The OAuth 2.0 Authorization Framework: Bearer Token Usage. <https://www.rfc-editor.org/rfc/rfc6750>
- IETF. RFC 7636 — Proof Key for Code Exchange by OAuth Public Clients (PKCE). <https://www.rfc-editor.org/rfc/rfc7636>
- IETF. RFC 7519 — JSON Web Token (JWT). <https://www.rfc-editor.org/rfc/rfc7519>
- IETF. RFC 9068 — JSON Web Token (JWT) Profile for OAuth 2.0 Access Tokens. <https://www.rfc-editor.org/rfc/rfc9068>
- IETF. RFC 9700 — Best Current Practice for OAuth 2.0 Security. <https://www.rfc-editor.org/rfc/rfc9700>
- FastAPI. Documentación oficial. <https://fastapi.tiangolo.com/>
- SQLAlchemy. Documentación oficial (2.0). <https://docs.sqlalchemy.org/en/20/>
- PyJWT. Documentación oficial. <https://pyjwt.readthedocs.io/>
- argon2-cffi. Documentación oficial. <https://argon2-cffi.readthedocs.io/>
- Pytest. Documentación oficial. <https://docs.pytest.org/>

19. Apéndices

19.1 Matriz de requisitos, evidencia y estado

Requisito	Implementación	Prueba	Evidencia	Estado
Authorization Code + PKCE	app/oauth/router.py	TestScenario1	EV-02..EV-04, capturas 01-04	PASS
Client Credentials	app/oauth/router.py::_grant_client_credentials	TestScenario2	EV-09a/b	PASS
401 sin token / inválido	app/api/deps.py	TestScenario3 , TestScenario4	EV-05a/b	PASS

Requisito	Implementación	Prueba	Evidencia	Estado
403 scope insuficiente	app/api/deps.py::require_scopes	TestScenario5	EV-07	PASS
Aislamiento entre usuarios	app/api/router.py (ownership)	TestScenario6	EV-08a..d	PASS
JWT: claims y validación estricta	app/security.py	TestJwtHardening	EV-04b	PASS
alg=none rechazado	app/security.py (algorithms=[...])	test_alg_none_token_rejected	ev10_pytest_results.txt	PASS
Hash de contraseñas/secreros	app/security.py (Argon2id)	-	EV-01	PASS
Swagger/OpenAPI accesible	app/main.py (self-hosted)	-	EV-11, captura 09	PASS
Pruebas automatizadas (38)	tests/	pytest -v	ev10_pytest_results.txt	PASS

Tabla 5. Matriz de trazabilidad requisito → implementación → prueba → evidencia.

19.2 Diagramas fuente

Las fuentes editables (Mermaid) de ambos diagramas están disponibles en docs/diagrams/component_diagram.mmd y docs/diagrams/sequence_diagram.mmd, y pueden regenerarse como PNG con @mermaid-js/mermaid-cli.